Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота №1
З дисципліни «Технології захисту інформації»
Тема: «Вивчення класичних шифрів»

Виконав:
студент групи АІ-234
Сіголаєв Олександр Владиславович
Мазурцев Ніколас Андрійович
Перевірили:
ас. Трояк К.Ю.
ас. Соловйова Д.В.

Одеса 2026

Мета роботи – вивчення основних принципів шифрування інформації, знайомство з широко відомими алгоритмами шифрування, набуття навичок їхньої програмної реалізації.

3. Реалізувати шифрування і розшифрування файлу з використанням методу биграмм. Ключове слово вводиться.

Код програми (GitHub):

Лістинг розробленої програми з коментарями

— BigramCipherEngine.swift:

```
//
//  BigramCipherEngine.swift
//  BigramCipher
//
//  Core cipher logic: text normalization, bigram construction,
//  and Playfair encryption/decryption using the three standard rules.
//
//  PURPOSE:
//  This file contains the heart of the Playfair bigram cipher algorithm.
//  It is responsible for three stages of processing:
//
//    Stage 1 — NORMALIZATION:
//      Raw input text is cleaned into cipher-ready uppercase letters.
//      Non-letter characters (spaces, punctuation, digits) are stripped.
//      The letter 'J' is replaced with 'I' to match the 25-letter alphabet.
//
//    Stage 2 — BIGRAM SPLITTING:
//      The normalized text is split into consecutive pairs of characters.
//      Unlike classic English Playfair, identical-letter pairs (e.g., "LL")
//      are NOT split with a filler. They are kept as valid bigrams and will
//      follow the same-column rule during encryption/decryption.
//      A filler character ('X') is only appended at the END if the text
//      has an odd number of characters.
//
//    Stage 3 — TRANSFORMATION:
//      Each bigram is encrypted or decrypted using one of three Playfair rules,
//      determined by the relative positions of its two characters in the matrix:
//
//
//      ┌──────────────┬──────────────────────┬──────────────────────┐
//      │ Condition    │ Encrypt              │ Decrypt              │
//      ├──────────────┼──────────────────────┼──────────────────────┤
//      │ Same row     │ Shift columns RIGHT  │ Shift columns LEFT   │
//      │ Same column  │ Shift rows DOWN      │ Shift rows UP        │
//      │ Rectangle    │ Swap column indices  │ Swap column indices  │
//      └──────────────┴──────────────────────┴──────────────────────┘
//
//      The "same column" rule also applies to identical letter pairs (e.g., OO),
//      since both characters occupy the same row AND same column — the column
//      check matches first, and each character shifts down (or up) by one row.
//
//  DESIGN:
//  All functions are static and fully parameterized — no stored state, no global
//  variables. Every piece of data needed for processing is passed as a parameter.
//
//  Created by Oleksandr Siholaiev on 01.09.2026.
//

import Foundation

/// Engine that performs Playfair bigram cipher operations.
```

```swift
///
/// Provides three stages of processing:
/// 1. **Normalization** — clean raw text into cipher-ready uppercase letters.
/// 2. **Bigram splitting** — pair characters into bigrams.
/// 3. **Transformation** — apply Playfair rules to encrypt or decrypt bigram pairs.
///
/// All methods are static and take all required data as parameters,
/// satisfying the structured programming criterion of no global variables.
struct BigramCipherEngine {

    // MARK: - Stage 1: Text Normalization

    /// Normalizes raw input text for Playfair cipher processing.
    ///
    /// The normalization pipeline:
    /// 1. Convert every character to uppercase.
    /// 2. Replace the excluded character ('J' → 'I').
    /// 3. Strip all non-ASCII-letter characters (spaces, punctuation, digits).
    ///
    /// **Important:** This is a lossy transformation — the original spacing,
    /// punctuation, and letter casing cannot be recovered after normalization.
    /// This is an inherent property of the Playfair cipher, not a bug.
    ///
    /// - **Parameters**:
    ///   - text: The raw input text from the file.
    ///   - replacedChar: The character to replace (e.g., 'J').
    ///   - replacementChar: The substitute character (e.g., 'I').
    /// - **Returns**: A clean uppercase string containing only valid alphabet letters.
    static func normalizeText(
        _ text: String,
        replacedChar: Character,
        replacementChar: Character
    ) -> String {
        var result = ""

        for char in text.uppercased() {
            if char == replacedChar {
                // Replace excluded character with its substitute (J → I)
                result.append(replacementChar)
            } else if char.isLetter && char.isASCII {
                // Keep only ASCII letters — this filters out digits, punctuation,
                // spaces, emoji, and any non-English Unicode characters
                result.append(char)
            }
            // All other characters (spaces, punctuation, digits) are silently
discarded
        }

        return result
    }

    /// Counts how many non-letter characters will be stripped during normalization.
    ///
    /// Used to display a warning to the user when their input contains characters
    /// that the Playfair cipher cannot preserve (spaces, punctuation, digits, etc.).
    ///
    /// - **Parameter** text: The raw input text before normalization.
    /// - **Returns**: The number of characters that will be removed during normalization.
    static func countStrippedCharacters(in text: String) -> Int {
        var count = 0
        for char in text {
            // Count characters that are NOT ASCII letters (and not whitespace-only
lines)
            if !(char.isLetter && char.isASCII) && !char.isNewline {
                count += 1
            }
        }
        return count
    }
```

```swift
// MARK: - Stage 2: Bigram Construction

/// Splits normalized text into sequential bigram pairs.
///
/// Characters are grouped strictly left-to-right: `[(0,1), (2,3), (4,5), ...]`.
///
/// **Key difference from classic Playfair:** Identical-letter pairs (e.g., `OO`)
/// are **NOT** split with a filler character. They remain as valid bigrams and
/// will follow the same-column Playfair rule during transformation.
/// This matches the lab specification.
///
/// If the text has an odd number of characters, the filler character is appended
/// to complete the final pair. For example: `"HELLO"` → `[HE] [LL] [OX]`.
///
/// - **Parameters:**
///   - text: The normalized text (uppercase ASCII letters only).
///   - fillerCharacter: Character to append if text length is odd (typically 'X').
/// - **Returns:** An array of character pairs representing the bigrams.
static func makeBigrams(
    from text: String,
    fillerCharacter: Character
) -> [(Character, Character)] {
    var characters = Array(text)

    // Pad with filler if the total character count is odd
    if characters.count % 2 != 0 {
        characters.append(fillerCharacter)
    }

    // Group into sequential pairs — two characters at a time
    var bigrams: [(Character, Character)] = []
    var index = 0
    while index < characters.count {
        let first = characters[index]
        let second = characters[index + 1]
        bigrams.append((first, second))
        index += 2
    }

    return bigrams
}

// MARK: - Stage 3: Encryption & Decryption

/// Processes an array of bigrams through the Playfair cipher rules.
///
/// For each bigram, the two characters' positions in the matrix are looked up,
/// and one of three rules is applied based on their geometric relationship:
///
/// **Rule 1 — Same Row:**
/// Both characters are in the same row of the matrix.
/// Each character is replaced by the character to its right (encrypt)
/// or left (decrypt), wrapping around to the beginning/end of the row.
/// ```
/// Example (encrypt): In row [C I P H E], pair (C, H) → (I, E)
/// ```
///
/// **Rule 2 — Same Column (including identical letters):**
/// Both characters are in the same column of the matrix.
/// Each character is replaced by the character below it (encrypt)
/// or above it (decrypt), wrapping around to the top/bottom of the column.
/// This rule also applies when both characters are identical (e.g., LL),
/// since they trivially occupy the same row AND column.
/// ```
/// Example (encrypt): In column [C R G O V], pair (C, G) → (R, O)
/// ```
///
/// **Rule 3 — Rectangle:**
/// The two characters form opposite corners of a rectangle in the matrix.
```

```
    /// Each character is replaced by the character in the same row but at the
    /// other character's column. This rule is symmetric — it works the same
    /// for both encryption and decryption.
    /// ```
    /// Example: Pair (H, K) forms a rectangle:
    ///   H is at (0,3), K is at (2,1)
    ///   H → grid[0][1] = I,  K → grid[2][3] = M
    ///   Result: (I, M)
    /// ```
    ///
    /// - Parameters:
    ///   - bigrams: Array of character pairs to transform.
    ///   - matrix: The Playfair key matrix used for coordinate lookups.
    ///   - colCount: Number of columns in the matrix (for modular arithmetic).
    ///   - rowCount: Number of rows in the matrix (for modular arithmetic).
    ///   - shiftStep: The offset magnitude for row/column shifts (typically 1).
    ///   - encrypt: `true` for encryption (positive shift), `false` for decryption (negative shift).
    /// - Returns: The transformed text as a `String`.
    static func processBigrams(
        _ bigrams: [(Character, Character)],
        matrix: PlayfairMatrix,
        colCount: Int,
        rowCount: Int,
        shiftStep: Int,
        encrypt: Bool
    ) -> String {
        var result = ""

        // Determine shift direction: positive for encrypt, negative for decrypt
        let direction = encrypt ? shiftStep : -shiftStep

        for (first, second) in bigrams {
            // Look up positions of both characters in the matrix
            guard let pos1 = matrix.position(of: first),
                  let pos2 = matrix.position(of: second) else {
                // Defensive fallback: if a character is not found in the matrix,
                // preserve it unchanged (this should never happen with proper
normalization)
                result.append(first)
                result.append(second)
                continue
            }

            let newFirst: Character
            let newSecond: Character

            if pos1.row == pos2.row {
                // ── RULE 1: Same Row ──
                // Shift each character's column index with wrapping.
                // Encrypt: shift right (+1), Decrypt: shift left (-1).
                // Adding `colCount` before modulo ensures no negative indices.
                let newCol1 = (pos1.col + direction + colCount) % colCount
                let newCol2 = (pos2.col + direction + colCount) % colCount
                newFirst = matrix.grid[pos1.row][newCol1]
                newSecond = matrix.grid[pos2.row][newCol2]

            } else if pos1.col == pos2.col {
                // ── RULE 2: Same Column (also handles identical letter pairs) ──
                // Shift each character's row index with wrapping.
                // Encrypt: shift down (+1), Decrypt: shift up (-1).
                let newRow1 = (pos1.row + direction + rowCount) % rowCount
                let newRow2 = (pos2.row + direction + rowCount) % rowCount
                newFirst = matrix.grid[newRow1][pos1.col]
                newSecond = matrix.grid[newRow2][pos2.col]

            } else {
                // ── RULE 3: Rectangle ──
                // Swap column indices while keeping the original row indices.
                // This operation is its own inverse — symmetric for encrypt and
decrypt.
                newFirst = matrix.grid[pos1.row][pos2.col]
```

```
                newSecond = matrix.grid[pos2.row][pos1.col]
            }

            result.append(newFirst)
            result.append(newSecond)
        }

        return result
    }

    // MARK: - Display Formatting

    /// Formats bigram pairs into a human-readable string for display.
    ///
    /// Each pair is shown in square brackets, separated by spaces.
    /// Example output: `[HE] [LL] [OW] [OR] [LD]`
    ///
    /// - Parameter bigrams: The array of character pairs to format.
    /// - Returns: A formatted string showing each bigram in brackets.
    static func formatBigrams(_ bigrams: [(Character, Character)]) -> String {
        return bigrams.map { "[\(String($0.0))\(String($0.1))]" }.joined(separator: "
")
    }

    /// Formats cipher output text into space-separated bigram groups for readability.
    ///
    /// Inserts a space after every two characters so the output mirrors the
    /// bigram structure. For example: `"CFSVSNAB"` → `"CF SV SN AB"`.
    ///
    /// - Parameter text: The encrypted or decrypted text string.
    /// - Returns: The text with a space inserted after every two characters.
    static func formatOutputText(_ text: String) -> String {
        var formatted = ""
        for (index, char) in text.enumerated() {
            formatted.append(char)
            // Insert a space after every second character (except at the very end)
            if index % 2 == 1 && index < text.count - 1 {
                formatted.append(" ")
            }
        }
        return formatted
    }
}
```

— CipherConfiguration.swift:

```
//
//  CipherConfiguration.swift
//  BigramCipher
//
//  Contains all named constants used throughout the Playfair bigram cipher
application.
//
//  PURPOSE:
//  This file serves as the single source of truth for every configurable parameter.
//  By centralizing constants here, we ensure:
//    - Zero "magic numbers" scattered across the codebase.
//    - Easy reconfiguration without modifying cipher logic.
//    - Full compliance with the structured programming requirement.
//
//  HOW THE PLAYFAIR ALPHABET WORKS:
//  The standard English Playfair cipher uses a 25-letter alphabet arranged in a 5×5
grid.
//  Since 26 letters don't fit evenly into a 5×5 matrix (25 cells), one letter must be
//  excluded. By convention, 'J' is merged with 'I' — any 'J' in the input text is
//  treated as 'I' before encryption. This is the most common variant used in
textbooks.
//
//  Created by Oleksandr Siholaiev on 01.09.2026.
```

```swift
//

import Foundation

/// Central configuration for the Playfair bigram cipher.
///
/// All parameters that control cipher behavior are defined here as named constants.
/// No numeric or character literals should appear in the cipher engine, matrix builder,
/// or file service — they should all reference values from this struct.
///
/// ## Example Usage
/// ```swift
/// let size = CipherConfiguration.matrixRowCount  // 5
/// let alpha = CipherConfiguration.alphabet        // "ABCDEFGHIKLMNOPQRSTUVWXYZ"
/// ```
struct CipherConfiguration {

    // MARK: - Alphabet Constants

    /// The 25-character English alphabet used to populate the Playfair matrix.
    ///
    /// The letter 'J' is excluded and normalized to 'I' (standard Playfair convention).
    /// This string defines the exact set of characters that can appear in the matrix
    /// and in cipher input/output.
    static let alphabet: String = "ABCDEFGHIKLMNOPQRSTUVWXYZ"

    /// The character that gets replaced during text normalization.
    ///
    /// In standard Playfair, 'J' is replaced with 'I' so the 26-letter English alphabet
    /// fits into a 25-cell (5×5) matrix. Any occurrence of this character in plaintext
    /// or keyword will be silently substituted before processing.
    static let replacedCharacter: Character = "J"

    /// The character that replaces `replacedCharacter` during normalization.
    ///
    /// Every 'J' becomes 'I'. This means "JULIA" and "IULIA" produce identical ciphertext,
    /// which is an intentional and well-known property of the Playfair cipher.
    static let replacementCharacter: Character = "I"

    // MARK: - Matrix Dimension Constants

    /// Number of rows in the Playfair matrix (5 for the standard English variant).
    ///
    /// Together with `matrixColCount`, determines the total matrix capacity:
    /// `matrixRowCount × matrixColCount = 25` cells, matching the 25-letter alphabet.
    static let matrixRowCount: Int = 5

    /// Number of columns in the Playfair matrix (5 for the standard English variant).
    static let matrixColCount: Int = 5

    // MARK: - Cipher Operation Constants

    /// The shift offset applied when two bigram characters share the same row or column.
    ///
    /// During encryption, characters are shifted by +shiftStep positions (right or down).
    /// During decryption, they are shifted by -shiftStep positions (left or up).
    /// Wrapping is applied via modular arithmetic so the shift "wraps around" the matrix.
    static let shiftStep: Int = 1

    /// Character appended to the end of normalized text when its length is odd.
    ///
    /// The Playfair cipher requires all text to be split into pairs (bigrams).
    /// If the normalized text has an odd number of characters, this filler character
    /// is appended to ensure the final character has a partner.
    ///
    /// Common choices are 'X' or 'Z'. We use 'X' as it is the standard convention.
    static let fillerCharacter: Character = "X"
}
```

— CryptoError.swift:

```swift
//
//  CryptoError.swift
//  BigramCipher
//
//  Defines custom error types for all failure scenarios in the cipher application.
//
//  PURPOSE:
//  Instead of allowing the program to crash with opaque Swift runtime errors,
//  we define specific error cases for every known failure mode. Each case carries
//  contextual data (e.g., the file path that failed) and produces a human-readable
//  message via `LocalizedError` conformance.
//
//  This approach satisfies the grading criterion:
//  "All possible errors and non-standard situations are handled by the program,
//   which produces a corresponding message."
//
//  Created by Oleksandr Siholaiev on 01.09.2026.
//

import Foundation

/// Enumeration of all recoverable errors that the cipher application can encounter.
///
/// Conforming to `LocalizedError` ensures each case produces a descriptive message
/// via the `errorDescription` property. This is used in `do-catch` blocks throughout
/// the application to print meaningful feedback to the user.
///
/// ## Error Cases Overview
/// | Case              | When it occurs                                          |
/// |-------------------|---------------------------------------------------------|
/// | `fileNotFound`    | Input file path does not exist on disk                  |
/// | `unreadableFile`  | File exists but cannot be read as UTF-8 text            |
/// | `writeFailed`     | Output file cannot be created or written to             |
/// | `emptyKeyword`    | User entered an empty or all-punctuation keyword        |
/// | `emptyInputFile`  | File has no letters left after stripping non-alpha      |
enum CryptoError: LocalizedError {

    /// The file at the given path does not exist on disk.
    ///
    /// Triggered when `FileManager.fileExists(atPath:)` returns `false`.
    /// Carries the original user-supplied path for the error message.
    case fileNotFound(path: String)

    /// The file exists but its contents cannot be decoded as UTF-8 text.
    ///
    /// This can happen with binary files (images, executables) or files
    /// encoded in a non-UTF-8 character set.
    case unreadableFile(path: String)

    /// Writing to the specified output path failed.
    ///
    /// Carries both the target path and the underlying OS error reason
    /// (e.g., "Permission denied", "No such directory").
    case writeFailed(path: String, reason: String)

    /// The user-supplied keyword is empty or contains no valid alphabet characters.
    ///
    /// For example, a keyword of "123!!!" would trigger this after normalization
    /// strips all non-letter characters.
    case emptyKeyword

    /// The input file, after normalization, contains no processable text.
    ///
```

```
    /// This occurs when the file is empty or contains only spaces/punctuation/digits
    /// with no actual letter characters to encrypt or decrypt.
    case emptyInputFile

    // MARK: - LocalizedError Conformance

    /// A human-readable description of the error, printed to the console on failure.
    ///
    /// Each message starts with "Error:" for consistent formatting in terminal output.
    var errorDescription: String? {
        switch self {
        case .fileNotFound(let path):
            return "Error: File not found at path '\(path)'."
        case .unreadableFile(let path):
            return "Error: Unable to read file at path '\(path)' as UTF-8 text."
        case .writeFailed(let path, let reason):
            return "Error: Failed to write to '\(path)'. Reason: \(reason)"
        case .emptyKeyword:
            return "Error: The keyword is empty or contains no valid alphabet
characters."
        case .emptyInputFile:
            return "Error: The input file contains no processable text after
normalization."
        }
    }
}
```

— FileService.swift:

```
//
//  FileService.swift
//  BigramCipher
//
//  Provides safe file read/write operations with comprehensive error handling.
//
//  PURPOSE:
//  All file system interactions in the application are routed through this service.
//  This ensures consistent error handling — every I/O failure is caught and reported
//  as a `CryptoError` with a descriptive message, rather than causing a crash.
//
//  DESIGN NOTES:
//  - Uses `FileManager` for existence checks and raw data reading.
//  - Supports tilde (~) expansion for home directory paths.
//  - All methods are static — no instance state is needed for file operations.
//  - The service does NOT modify file contents; it reads/writes raw strings as-is.
//    Text normalization is handled separately by `BigramCipherEngine`.
//
//  Created by Oleksandr Siholaiev on 01.09.2026.
//

import Foundation

/// Service responsible for reading and writing text files.
///
/// Uses `CryptoError` to report failures with descriptive messages
/// instead of allowing unhandled exceptions to crash the program.
///
/// ## Usage Example
/// ```swift
/// let text = try FileService.readFile(at: "~/Documents/input.txt")
/// try FileService.writeFile(content: "encrypted data", to: "output.txt")
/// ```
struct FileService {

    /// Reads the entire contents of a text file at the given path.
    ///
    /// The method performs two validation steps:
    /// 1. Checks that the file exists on disk (throws `.fileNotFound` if not).
    /// 2. Attempts to decode the file data as UTF-8 (throws `.unreadableFile` if it fails).
```

```
    ///
    /// Tilde (`~`) in paths is automatically expanded to the user's home directory,
    /// so both `"~/Desktop/file.txt"` and `"/Users/name/Desktop/file.txt"` work.
    ///
    /// - Parameter path: Absolute or relative path to the input file.
    /// - Returns: The file contents as a `String`.
    /// - Throws: `CryptoError.fileNotFound` if the file does not exist,
    ///           `CryptoError.unreadableFile` if it cannot be decoded as UTF-8.
    static func readFile(at path: String) throws -> String {
        let fileManager = FileManager.default

        // Expand tilde (~) for home directory paths (e.g., ~/Documents/...)
        let expandedPath = NSString(string: path).expandingTildeInPath

        // Verify the file exists before attempting to read it
        guard fileManager.fileExists(atPath: expandedPath) else {
            throw CryptoError.fileNotFound(path: path)
        }

        // Read raw bytes and attempt UTF-8 decoding
        guard let contents = fileManager.contents(atPath: expandedPath),
              let text = String(data: contents, encoding: .utf8) else {
            throw CryptoError.unreadableFile(path: path)
        }

        return text
    }

    /// Writes a string to a file at the given path, creating it if necessary.
    ///
    /// Uses atomic writing (`atomically: true`) to prevent partial writes —
    /// the content is first written to a temporary file, then renamed to the
    /// target path. This ensures the output file is never left in a corrupted state.
    ///
    /// - Parameters:
    ///   - content: The text content to write (e.g., encrypted or decrypted result).
    ///   - path: Absolute or relative path to the output file.
    /// - Throws: `CryptoError.writeFailed` if the write operation fails,
    ///           with the underlying OS error reason included in the message.
    static func writeFile(content: String, to path: String) throws {
        // Expand tilde for consistency with readFile
        let expandedPath = NSString(string: path).expandingTildeInPath

        do {
            try content.write(toFile: expandedPath, atomically: true, encoding: .utf8)
        } catch {
            throw CryptoError.writeFailed(path: path, reason:
error.localizedDescription)
        }
    }
}
```

— main.swift:

```
//
//  main.swift
//  BigramCipher
//
//  Entry point for the Bigram (Playfair) Cipher command-line tool.
//
//  PURPOSE:
//  This file provides the interactive menu-driven console interface.
//  It is the ONLY file that performs I/O (printing and reading from the console).
//  All cipher logic is delegated to the engine, matrix, and service modules.
//
//  WHAT THE USER SEES:
//  The program displays an interactive menu with options to encrypt, decrypt, or
exit.
//  For each operation, it prompts for a keyword and file paths, then displays:
```

```swift
//    1. The keyword and the generated 5×5 Playfair matrix
//    2. The original file text (as-is from the file)
//    3. The normalized text (uppercase, letters only)
//    4. The bigram pairs
//    5. The encrypted or decrypted result
//    6. Confirmation that the result was written to the output file
//
//  This satisfies the grading criterion for application visibility:
//  "Applications allow viewing and modifying keys, encrypted and decrypted texts
//   in all representations provided by the task."
//
//  Created by Oleksandr Siholaiev on 01.09.2026.
//

import Foundation

// MARK: - Helper Functions

/// Reads a line of input from the console, trimming leading/trailing whitespace.
///
/// This is the single point of user input reading in the application.
/// Using `readLine()` ensures the program works in any terminal environment.
///
/// - Parameter prompt: The text displayed to the user before reading input.
/// - Returns: The trimmed input string, or an empty string if input is unavailable.
func readInput(prompt: String) -> String {
    print(prompt, terminator: " ")
    return readLine()?.trimmingCharacters(in: .whitespacesAndNewlines) ?? ""
}

/// Prints a horizontal separator line to visually divide console output sections.
///
/// - Parameter width: The number of dash characters in the separator (default: 50).
func printSeparator(width: Int = 50) {
    print(String(repeating: "-", count: width))
}

// MARK: - Core Cipher Operation

/// Performs the full encrypt or decrypt workflow.
///
/// This function orchestrates the entire cipher pipeline:
/// 1. Prompts the user for a keyword, input file path, and output file path.
/// 2. Reads the input file via `FileService`.
/// 3. Builds the Playfair matrix from the keyword via `PlayfairMatrix`.
/// 4. Normalizes text and constructs bigrams via `BigramCipherEngine`.
/// 5. Applies encryption or decryption via `BigramCipherEngine.processBigrams`.
/// 6. Displays all intermediate representations for full visibility.
/// 7. Writes the result to the output file via `FileService`.
///
/// All errors are caught and reported with descriptive messages — the program
/// never crashes on invalid input or missing files.
///
/// - Parameter encrypt: `true` for encryption mode, `false` for decryption mode.
func performCipherOperation(encrypt: Bool) {
    let modeName = encrypt ? "Encryption" : "Decryption"

    print()
    printSeparator()
    print("  \(modeName) Mode")
    printSeparator()

    // --- Step 1: Collect user inputs at runtime (no hardcoded values) ---
    let keyword = readInput(prompt: "Enter keyword:")
    let inputPath = readInput(prompt: "Enter input file path:")
    let outputPath = readInput(prompt: "Enter output file path:")

    // --- Step 2: Validate inputs and execute cipher pipeline ---
    do {
        // Validate keyword: must contain at least one valid alphabet character
```

```
        let normalizedKeyword = BigramCipherEngine.normalizeText(
            keyword,
            replacedChar: CipherConfiguration.replacedCharacter,
            replacementChar: CipherConfiguration.replacementCharacter
        )
        guard !normalizedKeyword.isEmpty else {
            throw CryptoError.emptyKeyword
        }

        // Read the input file (may throw fileNotFound or unreadableFile)
        let rawText = try FileService.readFile(at: inputPath)

        // Build the Playfair matrix from the keyword
        let matrix = PlayfairMatrix(
            keyword: keyword,
            alphabet: CipherConfiguration.alphabet,
            rowCount: CipherConfiguration.matrixRowCount,
            colCount: CipherConfiguration.matrixColCount,
            replacedChar: CipherConfiguration.replacedCharacter,
            replacementChar: CipherConfiguration.replacementCharacter
        )

        // --- Display: Keyword and Matrix ---
        print()
        print("--- Keyword: \(keyword.uppercased()) ---")
        print()
        print("--- Playfair Matrix (\(CipherConfiguration.matrixRowCount)x\
(CipherConfiguration.matrixColCount)) ---")
        print(matrix.prettyPrint())
        print()

        // --- Display: Original text from the file ---
        print("--- Original Text ---")
        print(rawText)
        print()

        // Warn the user if non-letter characters were stripped during normalization
        let strippedCount = BigramCipherEngine.countStrippedCharacters(in: rawText)
        if strippedCount > 0 {
            print("Note: \(strippedCount) non-letter character(s) (spaces,
punctuation, digits)")
            print("      were removed during normalization. The Playfair cipher
operates")
            print("      only on letters — original formatting cannot be preserved.")
            print()
        }

        // Normalize the text (uppercase, J → I, strip non-letters)
        let normalizedText = BigramCipherEngine.normalizeText(
            rawText,
            replacedChar: CipherConfiguration.replacedCharacter,
            replacementChar: CipherConfiguration.replacementCharacter
        )

        // Validate that normalization produced at least one character
        guard !normalizedText.isEmpty else {
            throw CryptoError.emptyInputFile
        }

        // --- Display: Normalized text ---
        print("--- Normalized Text ---")
        print(normalizedText)
        print()

        // Split normalized text into bigram pairs
        let bigrams = BigramCipherEngine.makeBigrams(
            from: normalizedText,
            fillerCharacter: CipherConfiguration.fillerCharacter
        )

        // --- Display: Bigram pairs ---
        print("--- Bigrams ---")
```

```
            print(BigramCipherEngine.formatBigrams(bigrams))
            print()

            // Apply the Playfair cipher (encrypt or decrypt)
            let resultText = BigramCipherEngine.processBigrams(
                bigrams,
                matrix: matrix,
                colCount: CipherConfiguration.matrixColCount,
                rowCount: CipherConfiguration.matrixRowCount,
                shiftStep: CipherConfiguration.shiftStep,
                encrypt: encrypt
            )

            // --- Display: Result ---
            let resultLabel = encrypt ? "Encrypted" : "Decrypted"
            print("--- \(resultLabel) Text ---")
            print(BigramCipherEngine.formatOutputText(resultText))
            print()

            // Write the result to the output file
            try FileService.writeFile(content: resultText, to: outputPath)
            print("Result successfully written to '\(outputPath)'.")

        } catch let error as CryptoError {
            // Handle known cipher-related errors with descriptive messages
            print(error.errorDescription ?? "An unknown cipher error occurred.")
        } catch {
            // Handle any unexpected errors (e.g., OS-level issues)
            print("Unexpected error: \(error.localizedDescription)")
        }

        print()
    }

    // MARK: - Main Menu Loop

    /// Displays the main menu and routes user input to the appropriate operation.
    ///
    /// The loop continues until the user explicitly chooses to exit (option 3).
    /// Invalid inputs are handled gracefully with a prompt to re-enter.
    func runMainMenu() {
        // Easter egg 🥚 — a fun ASCII art banner displayed on startup
        let banner = """

        ╔═══════════════════════════════════════════════════════════════╗
        ║   🔐 "Dance like no one is watching. Encrypt like everyone is."   ║
        ╚═══════════════════════════════════════════════════════════════╝

        """

        // Display the banner on startup
        print(banner)

        var shouldContinue = true

        while shouldContinue {
            printSeparator()
            print("  Bigram (Playfair) Cipher Tool — Lab 1, Variant 3")
            printSeparator()
            print("1. Encrypt a file")
            print("2. Decrypt a file")
            print("3. Exit")

            let choice = readInput(prompt: "\nSelect an option (1-3):")

            switch choice {
            case "1":
                performCipherOperation(encrypt: true)
            case "2":
```

```
            performCipherOperation(encrypt: false)
        case "3":
            print("\nGoodbye! Stay encrypted. 🔐\n")
            shouldContinue = false
        default:
            print("\nInvalid option. Please enter 1, 2, or 3.\n")
        }
    }
}

// ═══════════════════════════════════════════════
//  Program Entry Point
// ═══════════════════════════════════════════════
runMainMenu()
```

— PlayfairMatrix.swift:

```
//
//  PlayfairMatrix.swift
//  BigramCipher
//
//  Constructs the Playfair 5×5 key matrix from a keyword and provides
//  coordinate lookup for individual characters within the grid.
//
//  HOW THE PLAYFAIR MATRIX IS BUILT:
//  The matrix is a 5×5 grid filled with all 25 letters of the English alphabet
//  (with J merged into I). The construction process is:
//
//    1. Take the keyword (e.g., "MONARCHY") and normalize it:
//       - Convert to uppercase
//       - Replace J with I
//    2. Remove duplicate letters, keeping only the first occurrence:
//       "MONARCHY" → M, O, N, A, R, C, H, Y (all unique, so nothing removed)
//    3. Fill the matrix left-to-right, top-to-bottom with keyword letters first,
//       then append the remaining unused alphabet letters in order:
//
//       M O N A R        ← keyword letters fill first
//       C H Y B D        ← 'B' is the first unused alphabet letter after keyword
//       E F G I K        ← continuing with remaining letters
//       L P Q S T
//       U V W X Z
//
//  The resulting matrix is the "key" for the entire cipher — both encryption
//  and decryption use the same matrix built from the same keyword.
//
//  Created by Oleksandr Siholaiev on 01.09.2026.
//

import Foundation

/// Represents the 5×5 Playfair key matrix built from a keyword.
///
/// This is a pure value type — its state is fully determined at initialization
/// and it holds no mutable or global references. The matrix can be inspected
/// via `prettyPrint()` and queried via `position(of:)`.
struct PlayfairMatrix {

    // MARK: - Properties

    /// The 2D grid of characters, indexed as `grid[row][col]`.
    ///
    /// For a 5×5 matrix, valid indices are `grid[0...4][0...4]`.
    /// Each cell contains exactly one unique character from the alphabet.
    let grid: [[Character]]

    /// Number of rows in the matrix (used for bounds checking and modular arithmetic).
    let rowCount: Int

    /// Number of columns in the matrix (used for bounds checking and modular arithmetic).
```

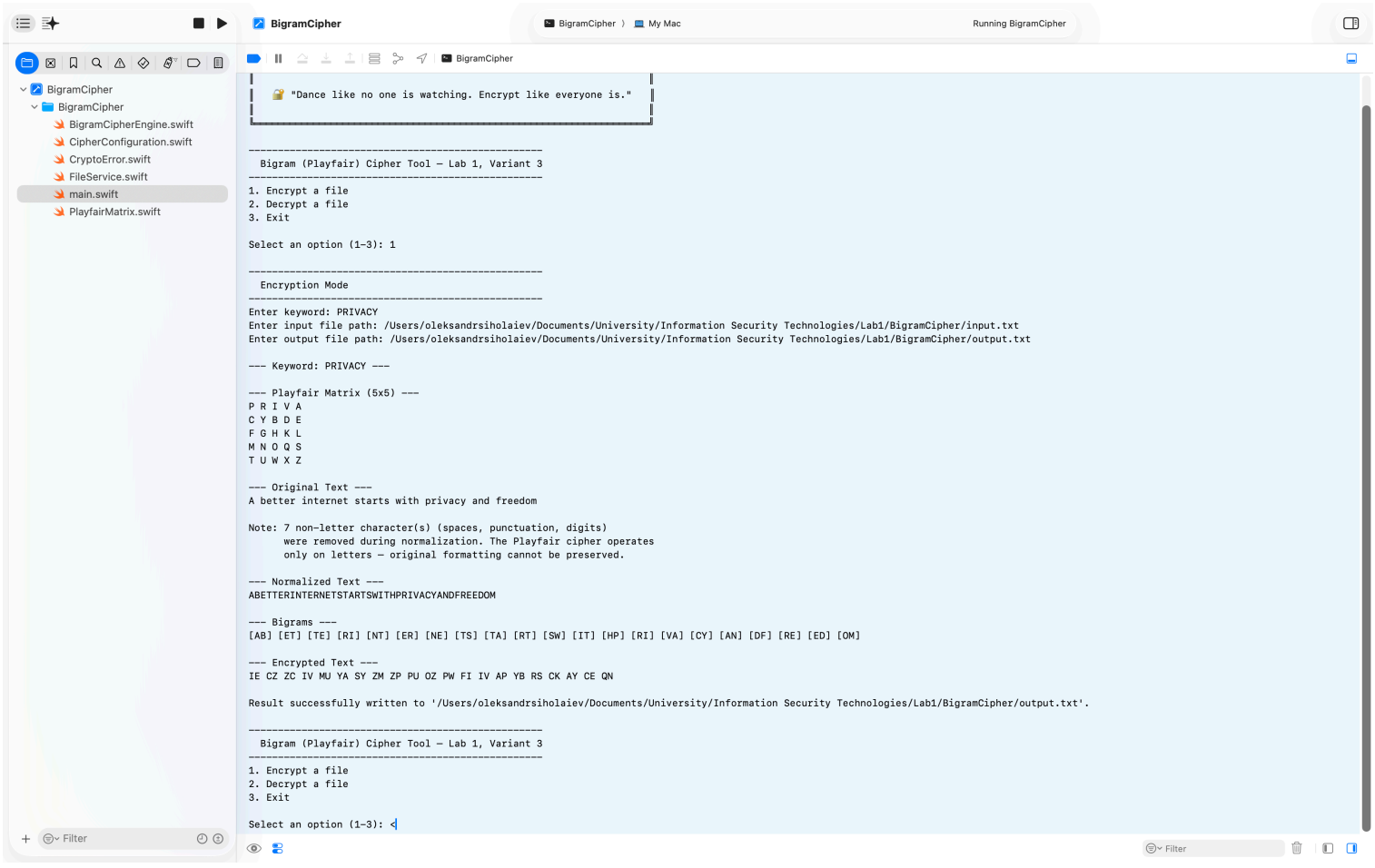
```swift
    let colCount: Int

    // MARK: - Initialization

    /// Creates a Playfair matrix from the given keyword and alphabet.
    ///
    /// The construction algorithm:
    /// 1. Normalize the keyword (uppercase, replace excluded character).
    /// 2. Walk through the normalized keyword and collect unique characters
    ///      (first occurrence only, preserving order).
    /// 3. Append remaining alphabet characters not already used by the keyword.
    /// 4. Arrange the resulting 25-character sequence into a 2D grid.
    ///
    /// - Parameters:
    ///   - keyword: The user-supplied encryption key (may contain duplicates, mixed case).
    ///   - alphabet: The full set of valid characters (25 English letters without J).
    ///   - rowCount: Number of rows in the grid (5 for standard Playfair).
    ///   - colCount: Number of columns in the grid (5 for standard Playfair).
    ///   - replacedChar: Character to replace during normalization ('J').
    ///   - replacementChar: Character to substitute in place of `replacedChar` ('I').
    init(keyword: String,
         alphabet: String,
         rowCount: Int,
         colCount: Int,
         replacedChar: Character,
         replacementChar: Character) {

        self.rowCount = rowCount
        self.colCount = colCount

        // Step 1: Normalize keyword — uppercase and replace excluded character (J → I)
        let normalizedKeyword = keyword.uppercased().map { char -> Character in
            return char == replacedChar ? replacementChar : char
        }

        // Step 2–3: Build ordered unique sequence — keyword chars first, then remaining
        var seen = Set<Character>()       // Tracks which characters have been placed
        var linearSequence: [Character] = [] // The final 25-character sequence

        // Add unique keyword characters (preserving their original order)
        for char in normalizedKeyword {
            if alphabet.contains(char) && !seen.contains(char) {
                seen.insert(char)
                linearSequence.append(char)
            }
        }

        // Fill remaining cells with alphabet characters not used by the keyword
        for char in alphabet {
            if !seen.contains(char) {
                seen.insert(char)
                linearSequence.append(char)
            }
        }

        // Step 4: Arrange the linear sequence into a 2D grid (row-major order)
        var matrix: [[Character]] = []
        for rowIndex in 0..<rowCount {
            let startIndex = rowIndex * colCount
            let endIndex = min(startIndex + colCount, linearSequence.count)
            let row = Array(linearSequence[startIndex..<endIndex])
            matrix.append(row)
        }

        self.grid = matrix
    }

    // MARK: - Coordinate Lookup
```

```swift
    /// Finds the row and column position of a character in the matrix.
    ///
    /// This is the core operation used by the cipher engine to determine
    /// which Playfair rule to apply (same row, same column, or rectangle).
    ///
    /// - Parameter character: The character to locate in the grid.
    /// - Returns: A tuple `(row, col)` if the character is found, or `nil` if
    ///            the character does not exist in the matrix.
    func position(of character: Character) -> (row: Int, col: Int)? {
        for row in 0..<rowCount {
            for col in 0..<colCount {
                if grid[row][col] == character {
                    return (row, col)
                }
            }
        }
        return nil
    }

    // MARK: - Display

    /// Returns a formatted multi-line string representation of the matrix,
    /// suitable for printing to the console.
    ///
    /// Each row is printed with characters separated by spaces.
    ///
    /// Example output for keyword "CIPHER":
    /// ```
    /// C I P H E
    /// R A B D F
    /// G K L M N
    /// O Q S T U
    /// V W X Y Z
    /// ```
    func prettyPrint() -> String {
        return grid.map { row in
            row.map { String($0) }.joined(separator: " ")
        }.joined(separator: "\n")
    }
}
```

input.txt

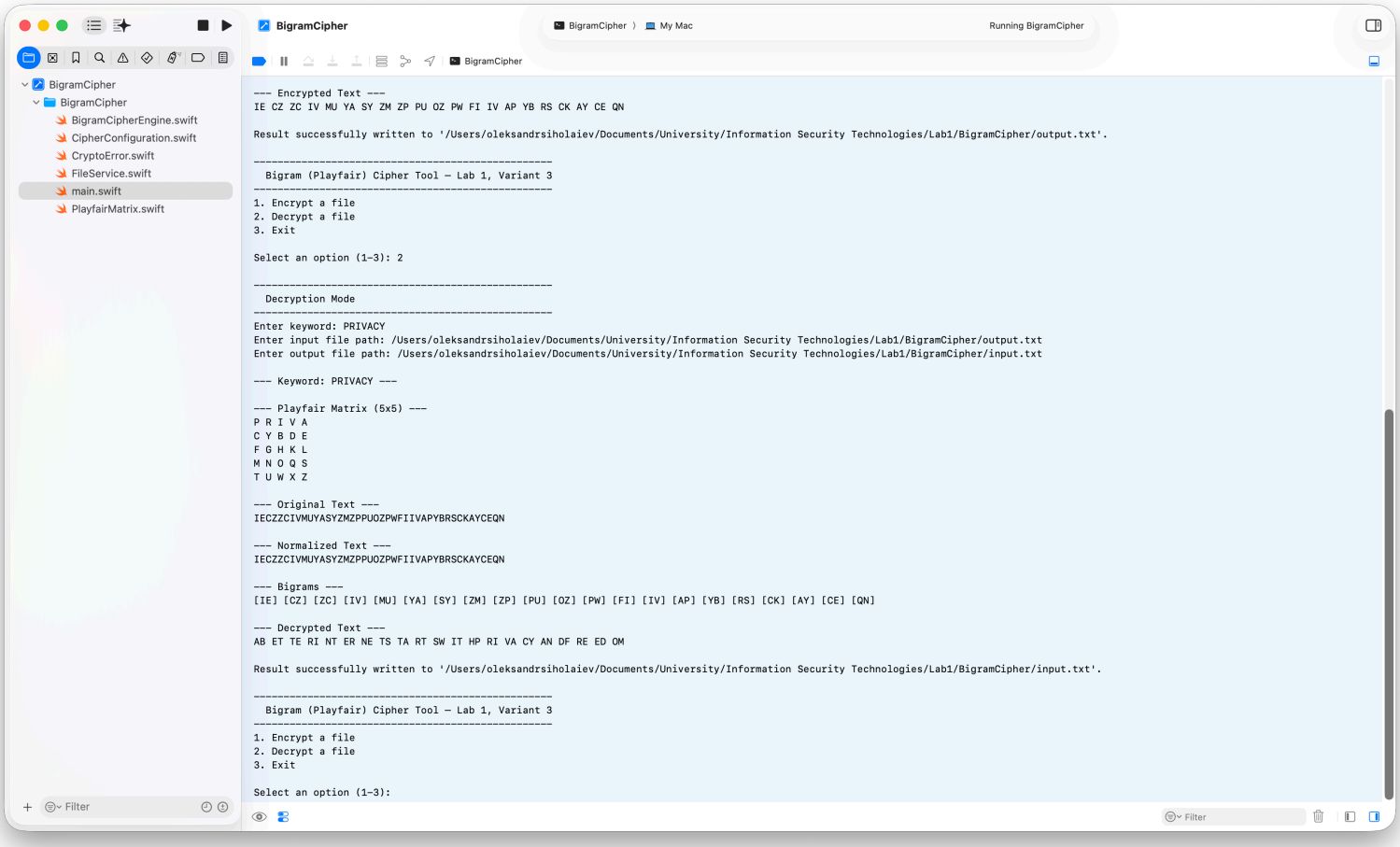
```
A better internet starts with privacy and freedom
```

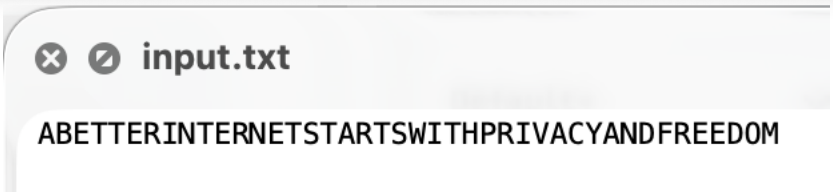
Результати роботи програми (encryption)



output.txt

```
IECZZCIVMUYASYZMZPPUOZPWFIIVAPYBRSCKAYCEQN
```





Результати роботи програми (decryption)